ABA PayWay Sandbox Integration

A matechnical guide for developers - Understanding payment flow, data structures, and webhook mechanisms

Contents

01. The Mailman Metaphor

Understanding the three key components: ABA Server, User Server, and Webhook mechanism.

02. Sandbox Workflow

Step-by-step payment processing flow from request to authentication and response handling.

03. Data Structures

Deep dive into Request, Response, Webhook, and Verification data structures with examples.

04. Security & Summary

Hash generation for securing transactions and understanding entity roles and responsibilities.

The Mailman Metaphor: Three Key Components

The Mailman
(ABA PayWay Server)

Processes payment requests in the sandbox environment

Delivers payment confirmations via webhook pushback

Acts as the central payment gateway

The House
(User Server)

Must be publicly accessible (not localhost)

POST method enabled to receive webhooks

Returns 200 OK status to acknowledge receipt

The Mailbox
(Webhook/callback_url)

Receives real-time payment status updates

Automatically pushed from ABA when payment events occur

Critical endpoint for payment confirmation

Sandbox Workflow: Payment Processing Steps

Step 1 - Request

Send payment request to Sandbox

API URL: https://checkout-sandbox.payway.com.kh/api/paymentgateway/v1/payments/purchase

Step 2 - Authentication

Use Sandbox Merchant ID and API Key obtained from the registration at

developer.payway.com.kh

Step 3 - Payment Simulation

Simulate payment by scanning QR with the test app or using Test Card Numbers

(Visa: 4111 1111 1111 1111)

Step 4 - Response

ABA sends a pushback webhook to your server confirming the payment status

Webhook Concepts & Ngrok Setup

Webhook vs API

Webhook uses Push mechanism server sends data immediately when event occurs.

Traditional API uses Polling you repeatedly request data. Webhooks enable real time notifications

The Localhost Challenge

- ABA Sandbox on public internet cannot reach localhost:3000
- The laptop is on private network without public address
- Like mailing a letter to a house without an address

Ngrok Solution

- Creates temporary public URL (https://xyz-123.ngrok-free.app)
- Tunnels public requests to your local development server
- Enables ABA to send webhooks during local testing

Request Data Structure: Generating QR Code Payment

When generating a QR code, send POST request to ABA PayWay Sandbox endpoint with the following fields:

Field	Type	Description
req_time	String	Current timestamp (YYYYMMDDHHMMSS)
merchant_id	String	Sandbox Merchant ID
tran_id	String	Unique Order ID (Max 20 chars)
amount	Decimal	Transaction amount (e.g., 10.50)
currency	String	USD or KHR
hash	String	HMAC-SHA512 security signature
type	String	Usually 'purchase'
payment_option	String	abapay_deeplink, wechat, or alipay
callback_url	String	Webhook URL for status updates

Response Data Structure: Receiving QR Code from ABA

After sending the request, ABA responds with QR data for displaying to the customer:


```
{
  "status": { "code": "0",
              "message": "Success",
              "trace_id": "65bdc1fde8eea..." },
  "qrString": "000201010212...",
  "qrImage": "...",
  "abapay_deeplink": "abamobilebank://...",
  "amount": 0.01,
  "currency": "USD"
                }
```

- qrString: Use this string to build a custom QR code for display or printing purposes
- qrImage: Base64 encoded PNG image that can be rendered directly in an tag or preview
- abapay_deeplink: Deep link URL for the Pay with ABA mobile button to trigger payment in app

Webhook (Pushback) Structure: Real-Time Notification

Most Important

This is sent to the callback_url once customer pays - Critical for updating in database!

{ "tran_id": "ORDER12345", "apv": "123456", "status": "0", "merchant_ref_no": "REF001", "hash": "a1b2c3d4..." }

1 **tran_id**

Matches the original Order ID

2 **apv**

Approval Code from bank(authorization confirmation)

3 **status**

0 = Success, 1 = Failed

4 **hash**

Security verification to confirm authenticity

Verification Structure: Manual Transaction Check

Webhooks can fail due to network issues. Always have a fallback mechanism to manually verify critical transactions by sending tran_id to ABA.


```
{
  "status": 0,
  "description": "Success",
  "amount": "10.00",
  "currency": "USD",
  "tran_id": "ORDER12345",
  "payment_status": "PAID",
  "payment_type": "ABA KHQR",
  "hash": "..."
}
```


Key Indicator

Look for payment_status: PAID - This confirms successful transaction.

Hash Generation: Securing The Integration

Purpose

Security signature to verify request authenticity and prevent data tampering during transmission.

Method

HMAC-SHA512 algorithm using the API Key as the secret signing key.

Process

Combine request fields into string following ABA's format, then sign using API Key to generate hash.

Security Note

Never expose API Key publicly. Always generate hash server-side, verify incoming webhook hash.

Summary: Entity Roles and Responsibilities

Entity	Role	Key Actions
System	Order Generator & Webhook Listener	Generates orders, listens for webhooks, updates database, returns 200 OK
ABA Sandbox	Payment Processor	Processes test transactions, pushes payment status to callback_url
The Customer	Payment Authorizer	Scans QR code, authorizes payment using test credentials
Webhook URL	Data Receiver	Mailbox where ABA delivers payment receipt and confirmation
Ngrok (Dev)	Local-to-Public Bridge	Tunnels local development server to public internet for testing

Thank You!